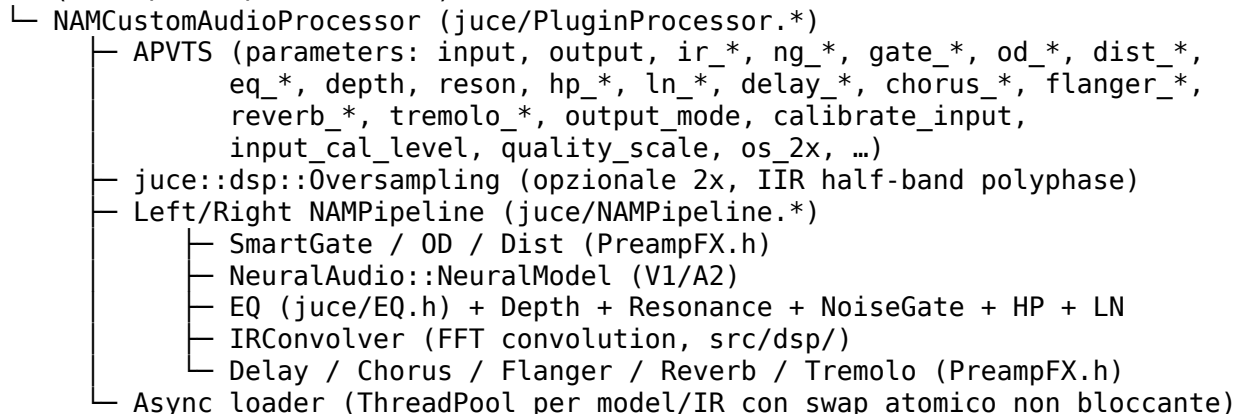


METAL NAM GEAR PLAYER — Guida Tecnica

Documentazione tecnica per utenti avanzati, tone-designer e sviluppatori.

Architettura

Host (VST3 / LV2 / Standalone)



Real-time safety

- **Hot path (processBlock) alloc-free:** allocazioni ammesse solo in prepare() e nei costruttori.
- **Load asincrono:** loadModelAsync / loadIRAsync fanno il parsing su un juce::ThreadPool e pubblicano il nuovo NAMPipeline in pendingL_/pendingR_ (raw ptr atomico). processBlock consuma lo swap non bloccante all'inizio del blocco.
- **Clear e prepare:** clearModel/clearIR e setOversamplingEnabled chiamano suspendProcessing(true/false) per evitare use-after-free con l'audio thread.
- **Modelli senza metadata:** la normalizzazione/calibrazione consulta i flag HasLoudness/HasInputLevel/HasOutputLevel esposti dal fork fabionet/NeuralAudio (branch nam-custom-patches); se assenti, nessuna correzione viene applicata.

Formato preset

.nampreset è XML (APVTS state) con l'aggiunta di:

- <Preset name="..." category="Clean|Rock|Metal|Extreme Metal|User">
- Riferimenti a model e ir (path o bundled:// per risorse embedded)

I preset di fabbrica sono embedded nel binario tramite juce_add_binary_data in juce/CMakeLists.txt. Aggiungere un .nampreset in juce/Presets/Factory/ richiede **anche** l'aggiunta alla lista SOURCES del binary data — altrimenti il file non viene incluso.

Gain-staging (porting Steve)

Tre parametri APVTS controllano la modalità di output:

- output_mode (Raw / Normalized / Calibrated, default Normalized)
- calibrate_input (bool, applica correzione input basata su metadata)
- input_cal_level (float, dBu, default 12.0)

Logica:

Modello ha	Modalità richiesta	Comportamento
loudness + input/output_level	Calibrated	Correzione pre-model + post-model in dBU
solo loudness	Normalized	Compensazione loudness ~14 dBU di headroom
loudness + input_level (metà)	Normalized	Fallback 50/50: metà correzione pre-model, metà post-model
nessun metadata	qualunque	No-op — il modello esce al suo livello nativo

Motivo del fallback 50/50: senza output_level_dbu non sappiamo dove sta il DAC virtuale del modello; splittare la compensazione evita saturazione dell'IR e self-noise amplificato a valle.

Modelli A2 (SlimmableContainer)

Rilevati via NeuralAudio (version > 0.5.4 o architecture == "SlimmableContainer" nel JSON del modello). Espongono:

- HasQualityScaling() — bool
- SetQualityScaleFactor(float 0..1) — RT-safe, pushata dall'editor via APVTS quality_scale

Lo slider SLIM sotto il loader NAM e il pomello QUAL sono bindati allo stesso parametro; un juce::Timer (8 Hz) grigia/attiva i due controlli in base a isCurrentModelSlimmable().

IR loader

- Formato: WAV mono o multicanale (viene tenuto solo il canale 0).
- Sample rate sorgente: bounded ($0 < \text{srcRate} \leq 768 \text{ kHz}$ per rifiutare header forgiati).
- Cap frame: $\min(\text{totalPCMFrameCount}, \text{srcRate} \times 4)$ prima di allocare (evita OOM da header malevolo).
- Cap finale IR: ~1.5 s @ target sample rate.
- Lettura via drwav_init_file streaming (non slurp completo), drwav_uninit garantito su ogni path (incluso bad_alloc).

FX chain — implementazione

Tutti gli FX in juce/PreampFX.h sono classi **header-only, framework-independent** (nessun include JUCE). Ogni classe segue lo schema:

```
struct XxxFX {
    void prepare(double sampleRate, int blockSize); // alloca qui
    float process(float x);                        // hot path, alloc-free
};
```

Nuovi effetti vanno aggiunti nello stesso stile (memoria team nam-juce-preampfx-framework-independent).

Build

Requisiti Linux: cmake ≥ 3.15, GCC/Clang C++17, ALSA, X11.

```
git clone --recurse-submodules https://github.com/fabionet/metal-nam-gear-player.git
cd metal-nam-gear-player
cmake -B build-juce -S . -DCMAKE_BUILD_TYPE=Release
```

```
cmake --build build-juce -j 2
rsync -a --delete "build-juce/juce/NAMCustom_artefacts/Release/VST3/NAM Custom.vst3/" "$HOME/
```

Note:

- **-DCMAKE_BUILD_TYPE=Release obbligatorio:** senza flag esplicito il tipo di build è vuoto → binario non ottimizzato → audio a scatti.
- **-j 2 massimo su macchine con poca RAM:** -j illimitato può causare OOM-kill (exit 137).
- Il target NAMCustom_LV2 produce NAM Custom.lv2/, il target NAMCustom_Standalone produce l'app JACK/ALSA.

Sicurezza (dalla v0.1.x)

- .nam corrotti/malevoli: nlohmann::json throw catturato → std::terminate dell'host evitato.
- WAV IR forgiati: cap frame + srcRate limitato + reject channels==0/>64 → OOM DoS mitigato.
- Use-after-free UI/audio in setOversamplingEnabled e clearModel/clearIR → risolti con suspendProcessing.

Licenze

- **Opera combinata: AGPL-3.0-or-later.** L'upstream mikeoliphant/neural-amp-modeler-lv2 è GPL-3.0, ma JUCE 8 è **AGPLv3 / commerciale:** per GPL-3 §13 il derivato combinato viene distribuito AGPL-3.0-or-later (upgrade permesso, downgrade no).
- Motore **NeuralAudio:** MIT (compatibile).
- Framework **JUCE 8:** AGPLv3 / commerciale (Raw Material Software Ltd.).
- **VST3 SDK:** dual GPLv3 / proprietary Steinberg; usato sotto il ramo GPLv3 (compatibile con l'AGPLv3 risultante).
- Asset preset demo bundled: MIT.

Trademark: "JUCE" è marchio di Raw Material Software Ltd., "VST" di Steinberg Media Technologies GmbH, "Neural Amp Modeler" di Steven Atkinson. Il progetto non è affiliato con nessuno dei suddetti.

Contribuire

Issues e PR su <https://github.com/fabionet/metal-nam-gear-player>. Per il submodule NeuralAudio con le patch delle Has-flags, PR contro <https://github.com/fabionet/NeuralAudio> branch nam-custom-patches.